AI-Based Presentation Skills Evaluation System

Supervisor: Dr. Fatma Helmy
Supervising Assistant: Eng. Saja Saadoun
Team Members: Ahmed Yehia, Arwaa Mamdoh, Mayar Adel, Mostafa Wael

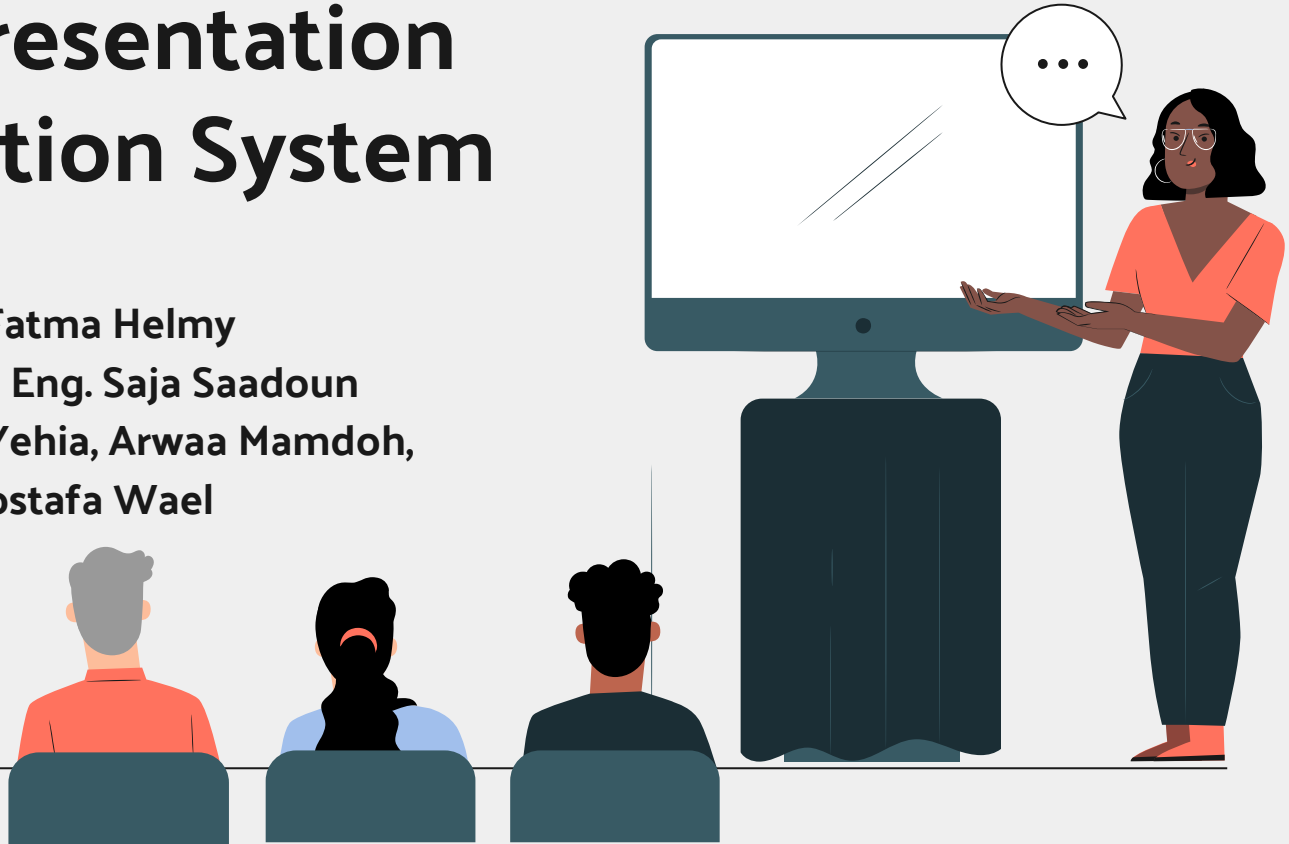


Table of contents

01 INTRODUCTION

02 REQUIREMENTS

03 METHODOLOGY

04 DESIGN

05 USER INTERFACE

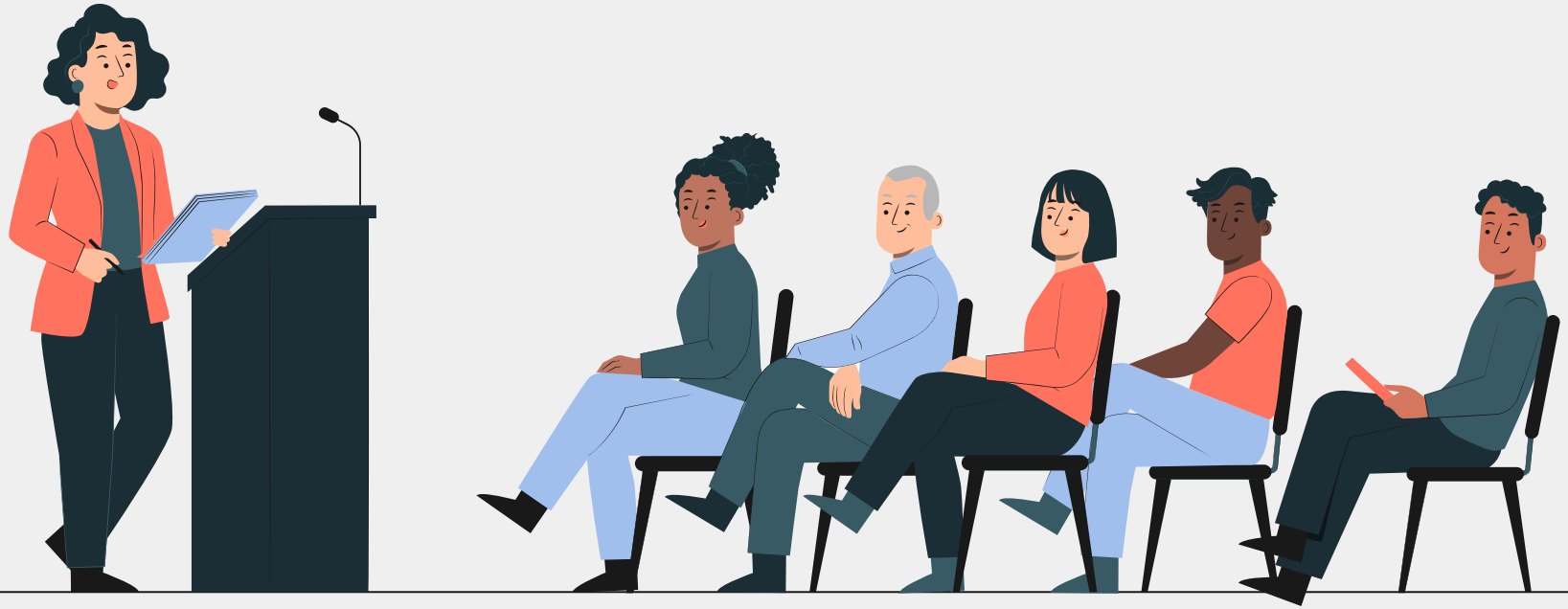
06 WHAT'S NEXT

07 DEMO

08 REFERENCES

09 APPENDIX

01 INTRODUCTION



01 INTRODUCTION

Problem Statement

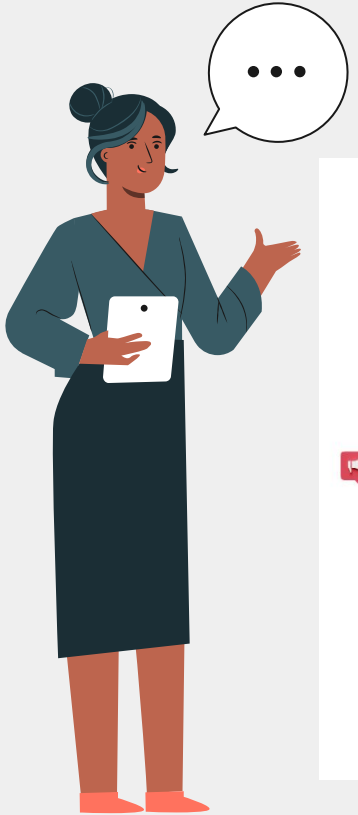
- Many individuals struggle with public speaking due to unclear speech, lack of confidence, and poor body language.
- Traditional training lacks objective feedback and is not scalable. Our system automates the evaluation of presentation skills, offering real-time AI-driven insights to improve performance.

01 INTRODUCTION

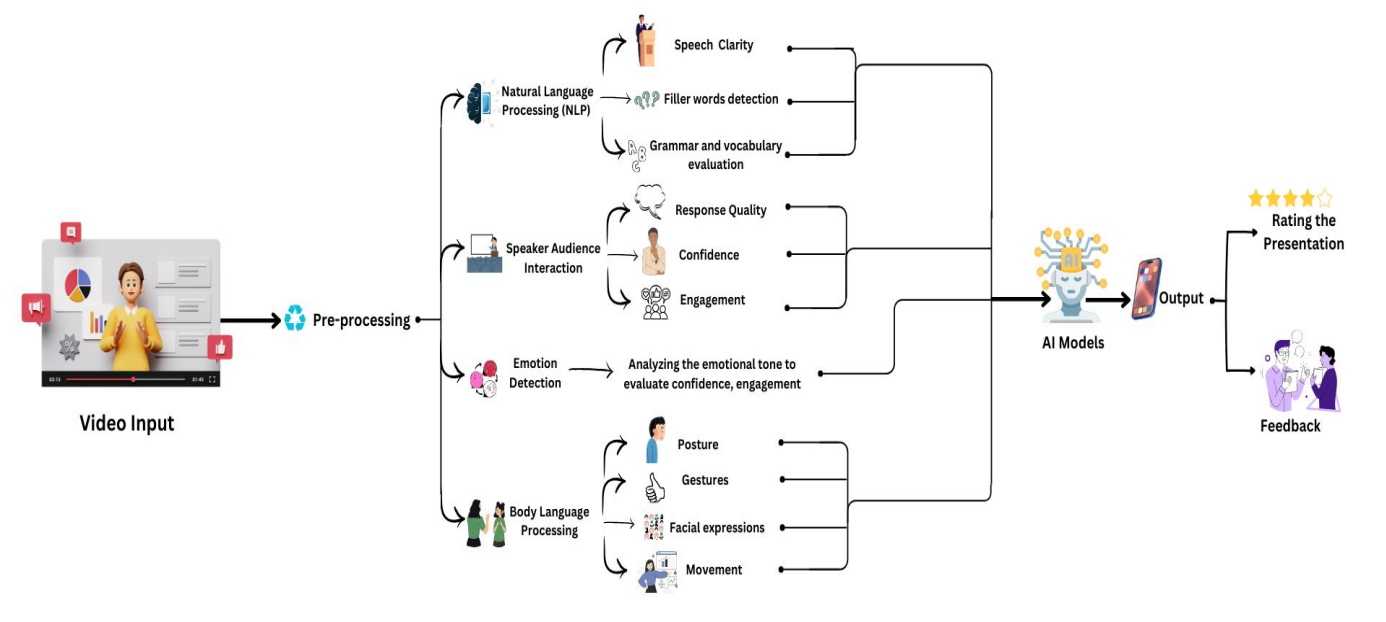
Objectives

- Analyze speech clarity, pace, and grammar.
- Detect body language, gestures, and facial expressions.
- Provide personalized feedback to improve public speaking skills.
- Track user progress over time.

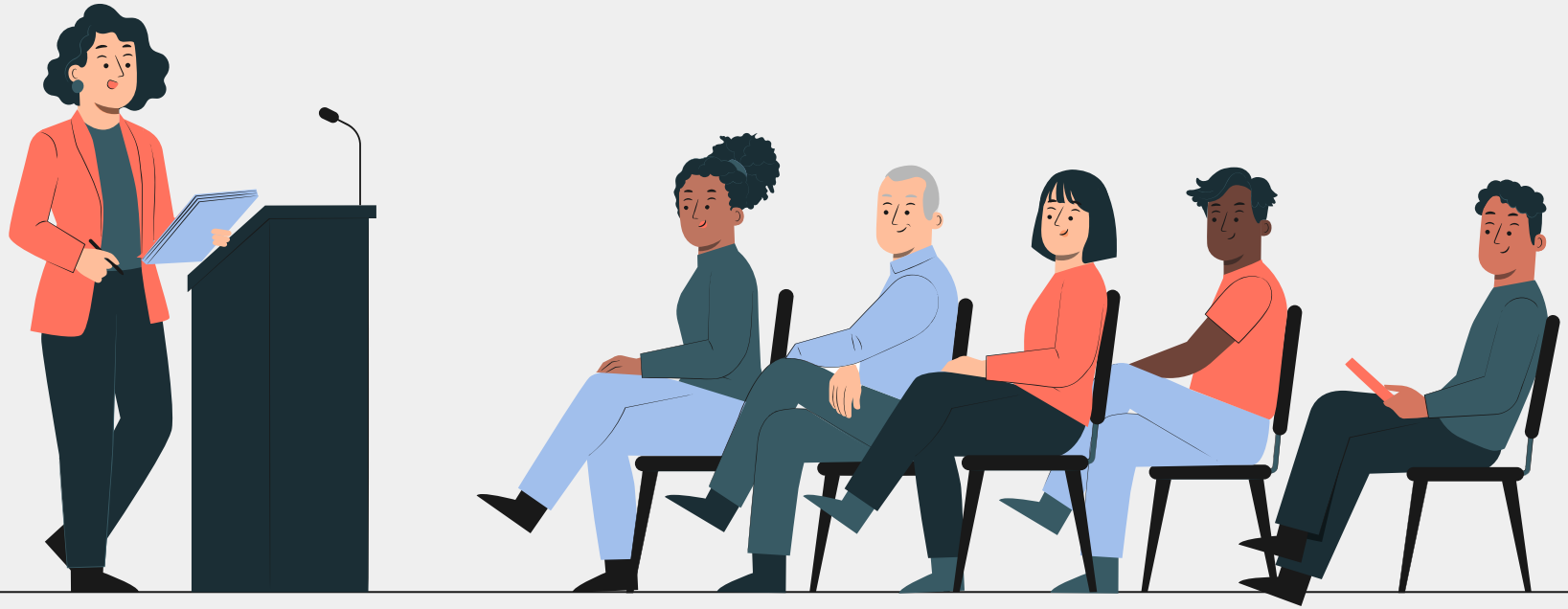
01 INTRODUCTION



System Overview



02 REQUIREMENTS

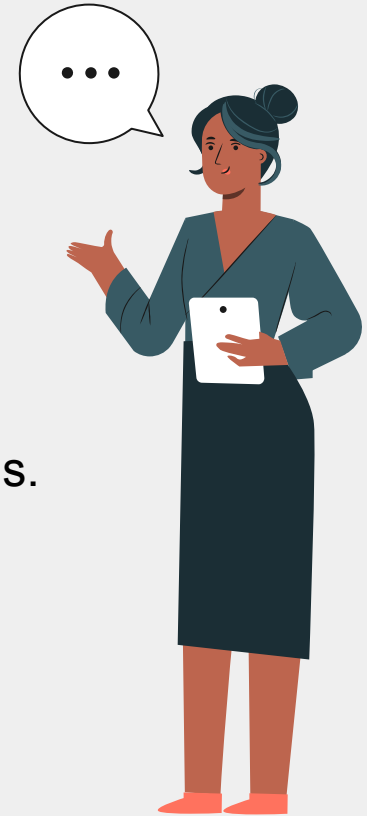


02 REQUIREMENTS

Functional Requirements

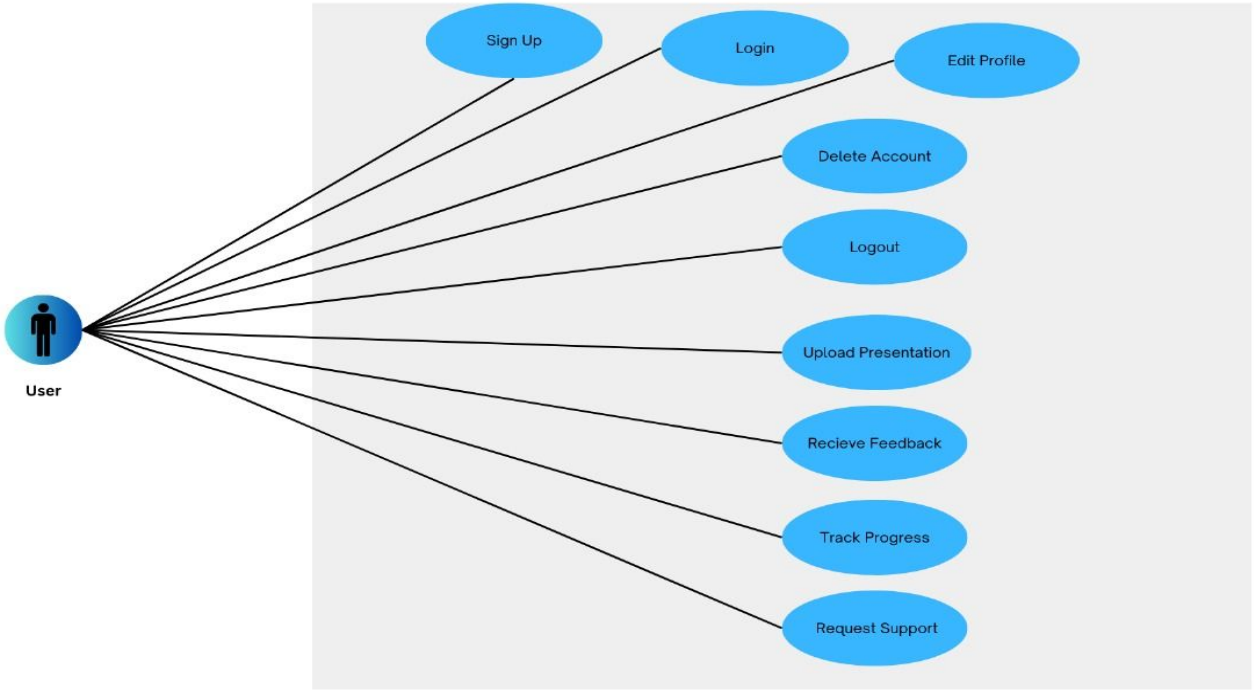
The system must:

- Allow users to upload presentation videos.
- Process speech, body language, and facial expressions.
- Generate feedback reports with improvement suggestions.
- Support multiple users and history tracking.



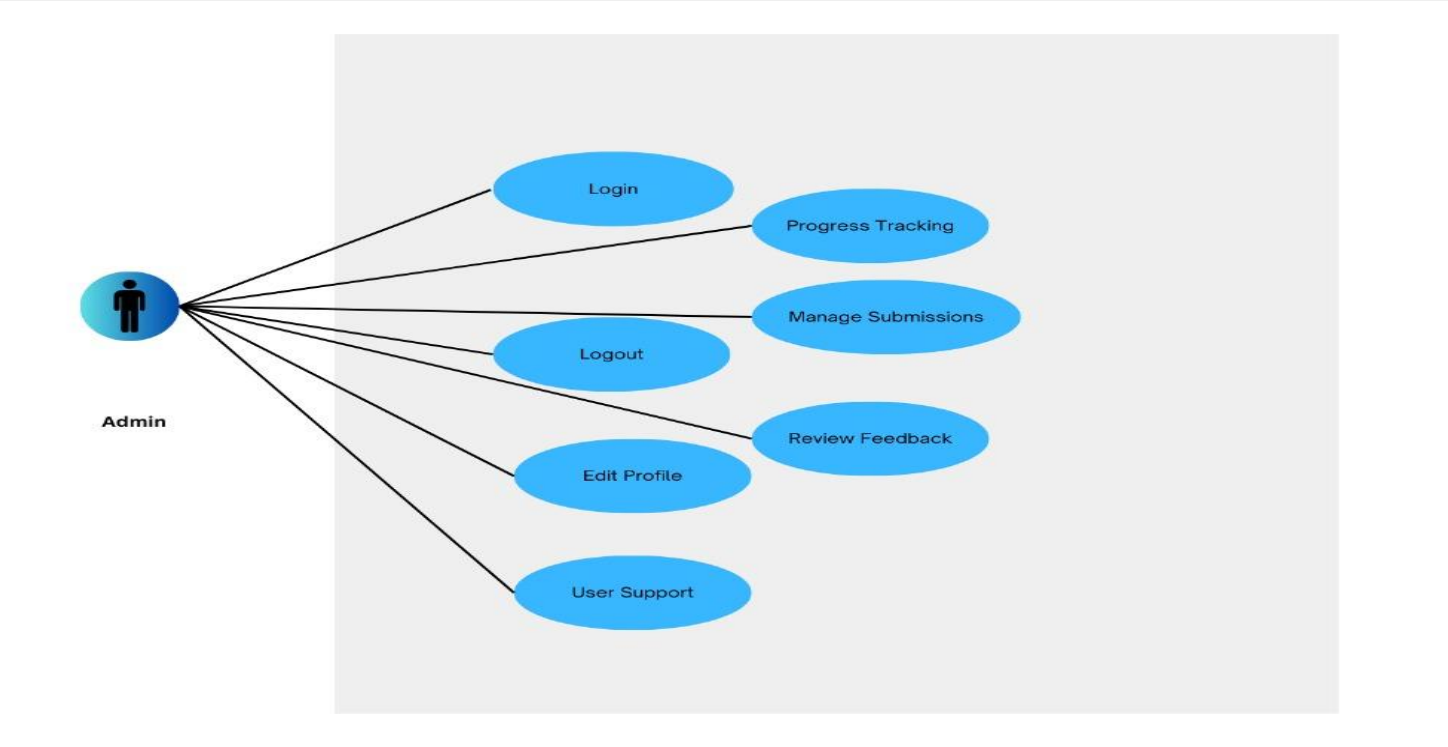
02 REQUIREMENTS

User Use Case Diagram



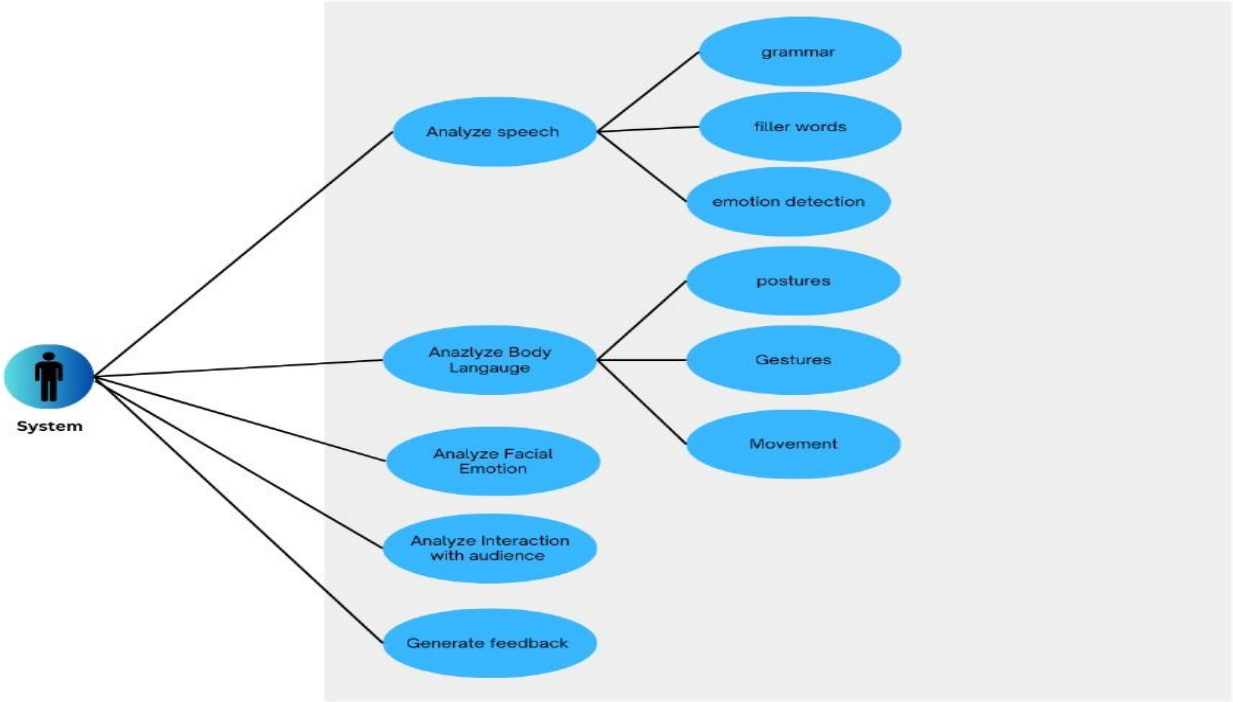
02 REQUIREMENTS

Admin Use Case Diagram



02 REQUIREMENTS

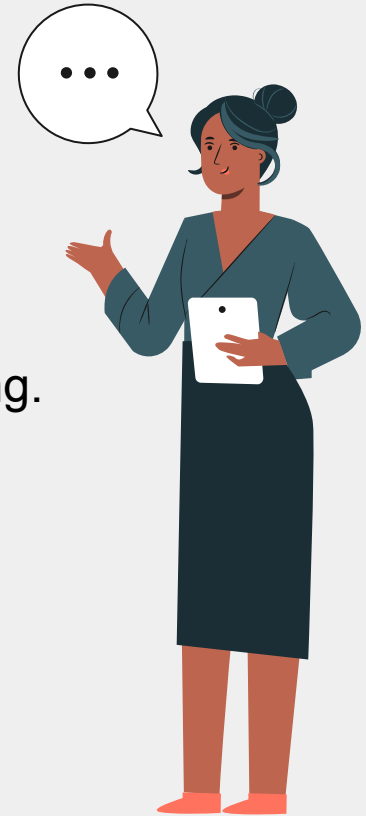
System Use Case Diagram



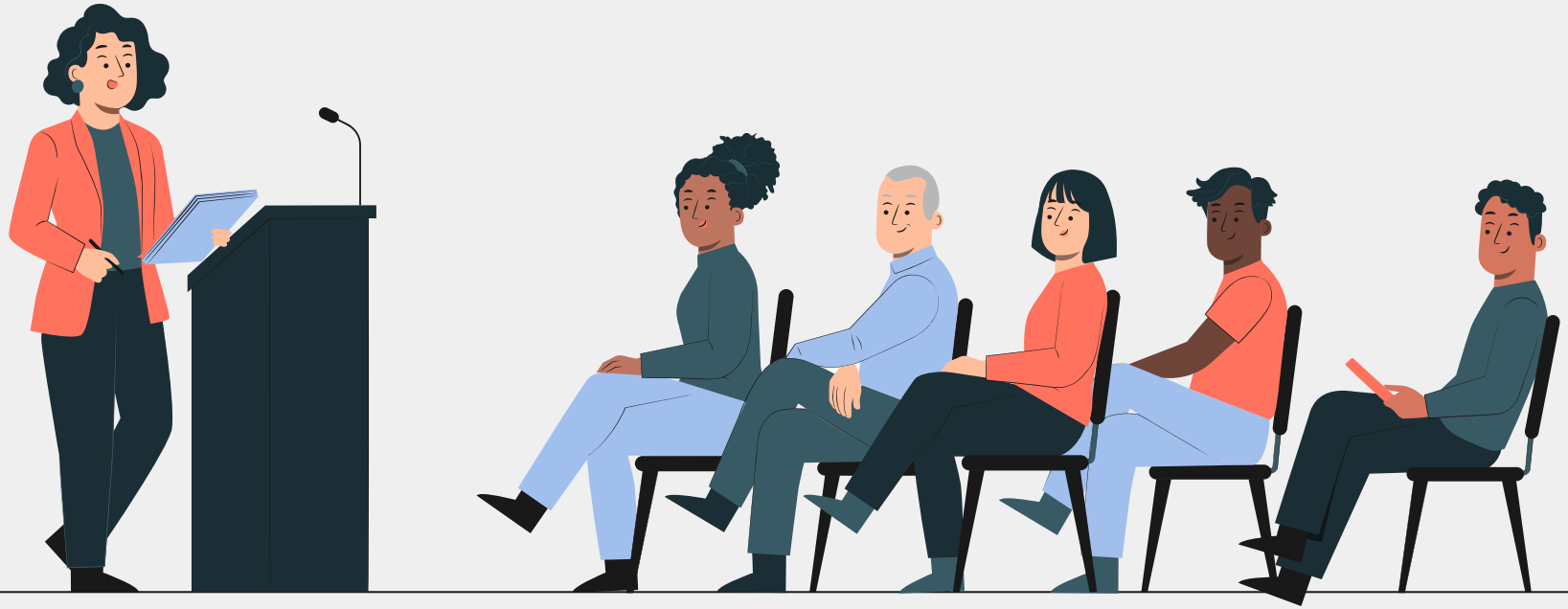
02 REQUIREMENTS

Non-Functional Requirements

Security: Use hashed passwords & role-based access control.
Maintainability: Regular performance monitoring & error logging.
Usability: User-friendly, intuitive interface.
Accessibility: Fully responsive across devices.
Extensibility: Scalable design for future enhancements.



03 METHODOLOGY



03 METHODOLOGY

Algorithms used :

- **Body Language (postures):** PoseNet
  - Captured body keypoints (head, shoulders, hips, etc.)
  - Analyzed alignment to classify postures (e.g., Slouching, Head Up)
  - Mapped postures to engagement levels (e.g., "Leaning Forward" → Interest)

- **Body Language (gestures):** MediaPipe
  - Captured video input
  - Detected hand landmarks (21 key points per hand)
  - Classified gestures based on finger positions (e.g., Open Palm, Thumbs Up)

03 METHODOLOGY

Algorithms used

- **Facial expressions:** Using VGG19 as a base model with added convolutional layers.

- **Tone Detection:** The model is a CNN-LSTM architecture, combining Convolutional Neural Networks (CNNs) for feature extraction and Long Short-Term Memory (LSTM) networks for sequential pattern learning in audio data.

03 METHODOLOGY

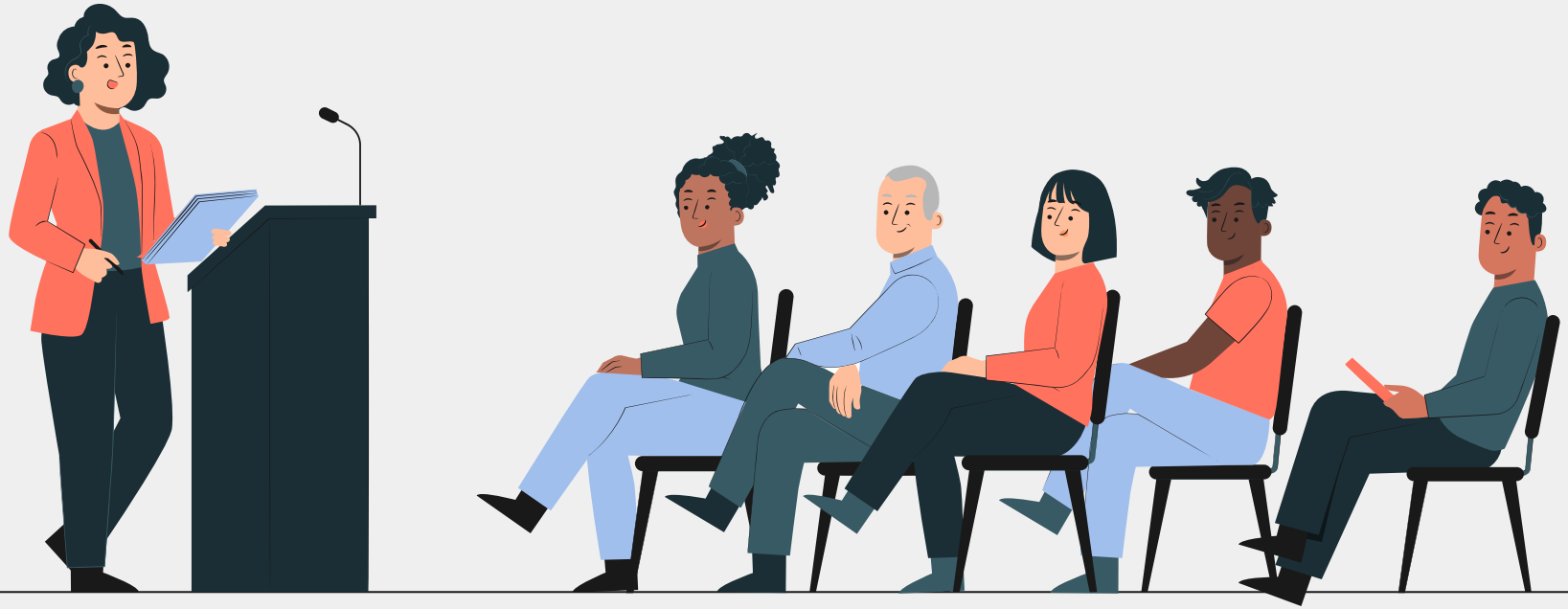
Algorithms used

- **Grammar & Pace:**The Google speech api is used to transcribe from audio to text with timestamps and then the transcription is passed into the t5-base-grammar-correction model.

- **Filler Words:** Facebook wave2vec base model is fine tuned with the podcast fillers dataset which is used to detect filler words.

- **Pronunciation:** A fine-tuned version of wave2vec is used to analyze speech and return an appropriate score.

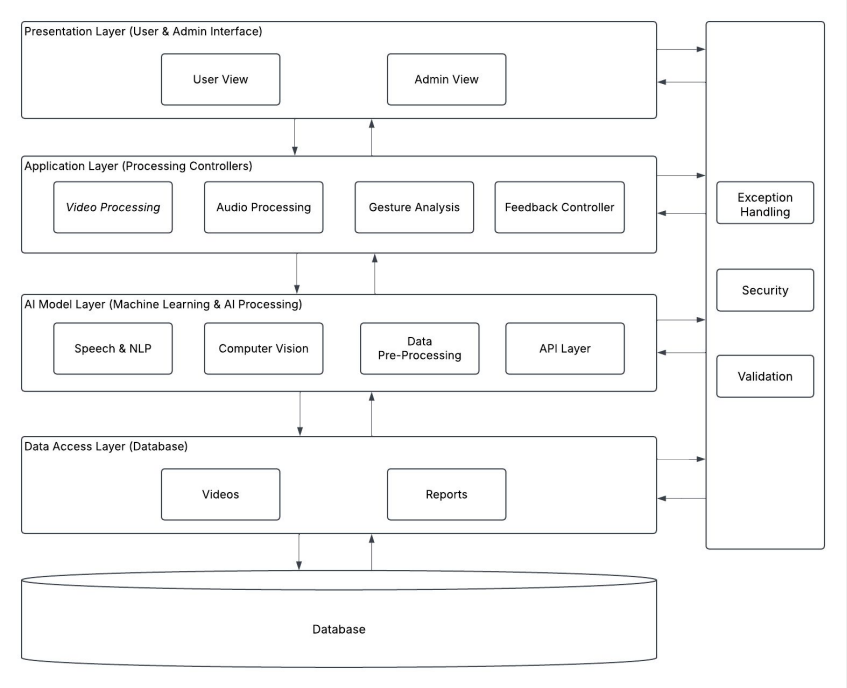
04 DESIGN



04 DESIGN

Architectural Diagram

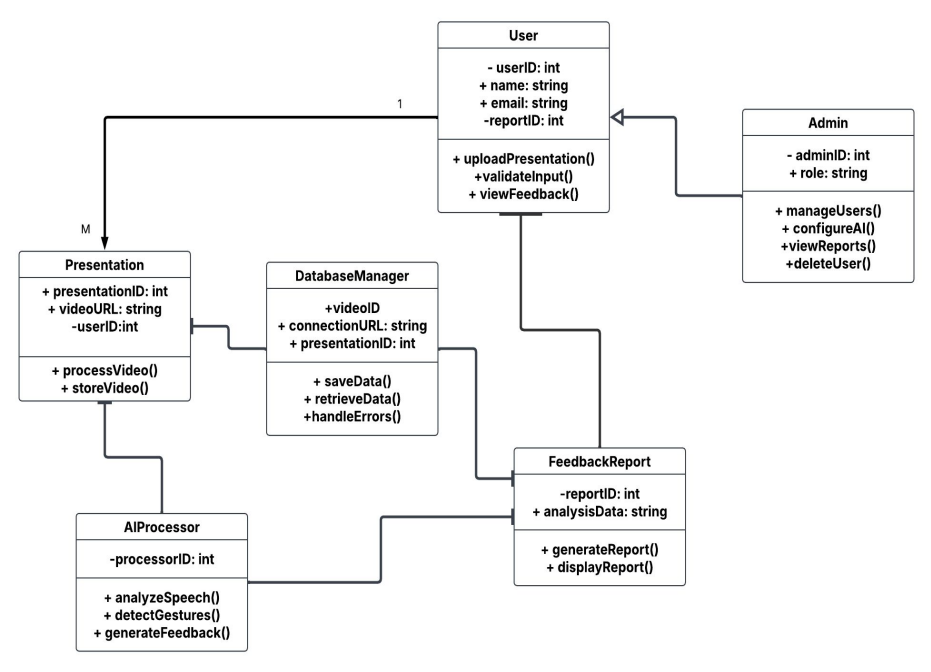
- Layered design ensuring modularity and scalability.
- Layers:
- Presentation Layer: User/Admin interfaces.
- Application Layer: Processes video, audio, gestures, and feedback.
- AI Model Layer: NLP, computer vision, and AI processing.
- Data Layer: Manages video and report storage.
- Security, validation, and exception handling enhance system reliability.



04 DESIGN

Class Diagram

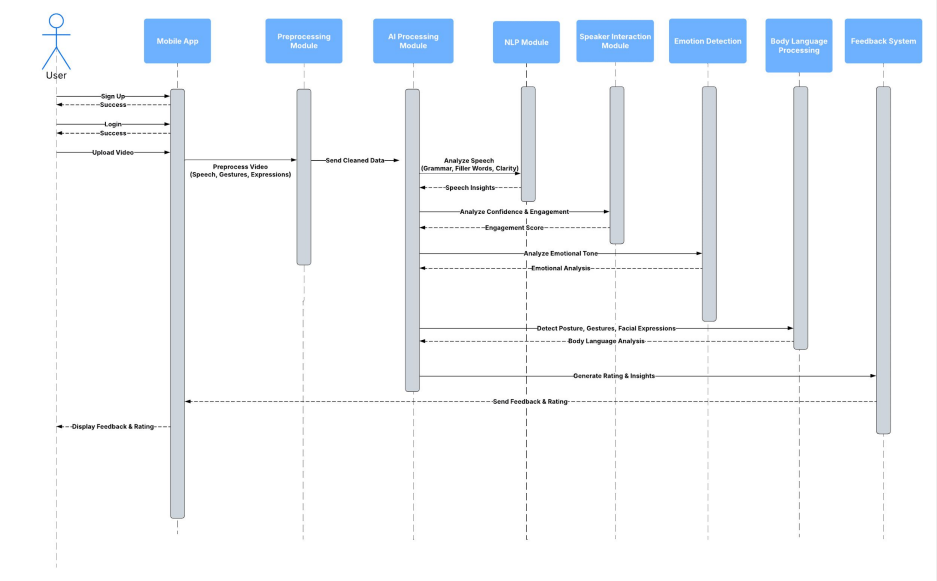
- User: Uploads presentations and views feedback.
- Admin: Manages users and configures AI settings.
- Presentation: Stores video details and processes uploaded files.
- DatabaseManager: Handles data storage and retrieval.
- AIProcessor: Analyzes speech, gestures, and generates feedback.
- FeedbackReport: Generates and displays the final evaluation report.



04 DESIGN

Sequence Diagram

- User signs up, logs in, uploads video.
- Preprocessing cleans data.
- AI analyzes speech, gestures, emotions, and body language.
- Feedback system generates insights.
- User receives evaluation report and ratings.



04 DESIGN

Entity-Relationship Diagram (ERD)

User table:

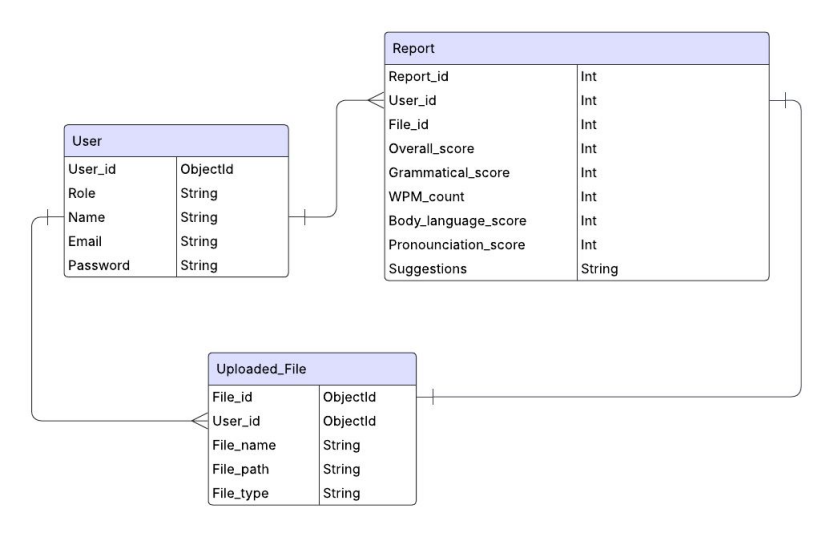
- Stores user information
- Users can upload files and receive reports

Uploade_File table:

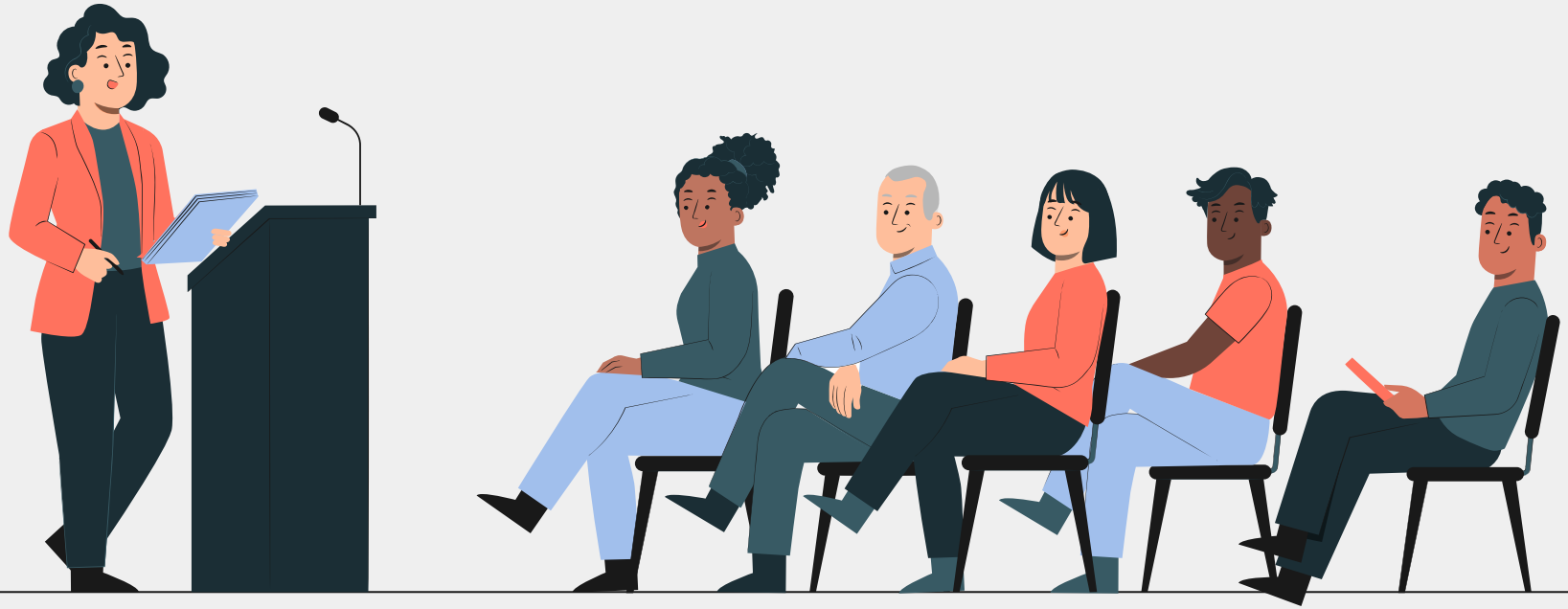
- Stores File information
- Each file is linked to a user and is analyzed for speech report.

Report:

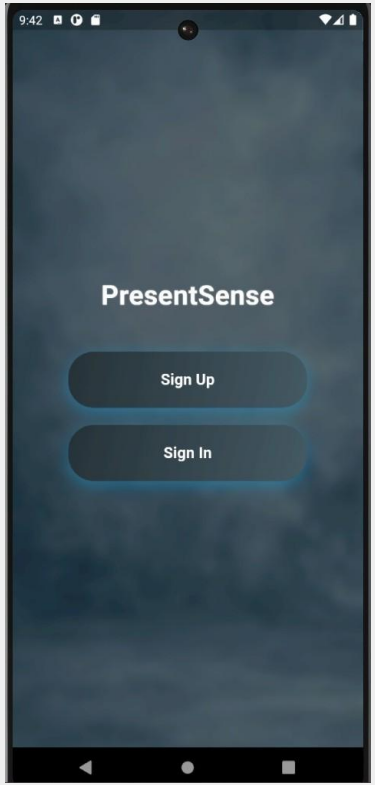
- Stores scores and report details.
- Each report is linked to a user and a specific Uploaded_File

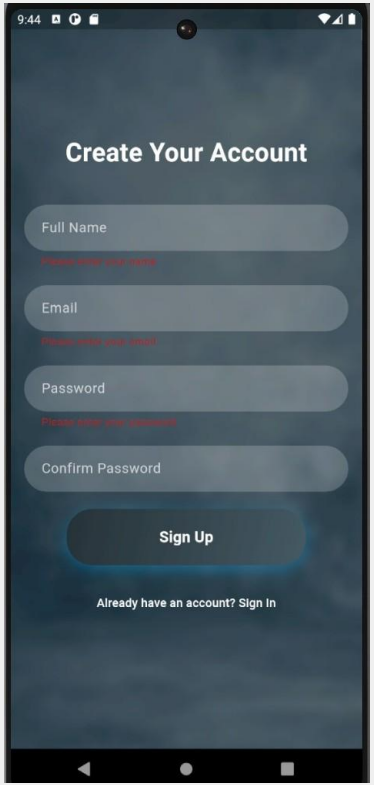


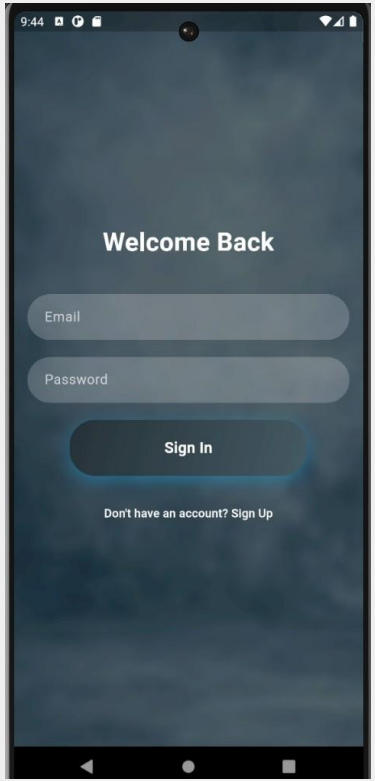
05 USER INTERFACE

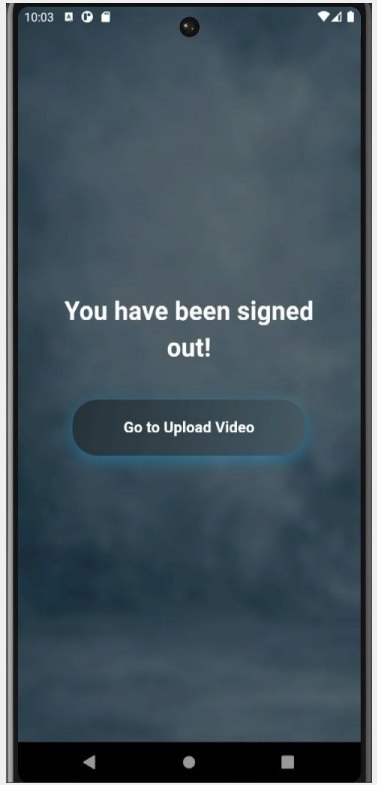


05 USER INTERFACE

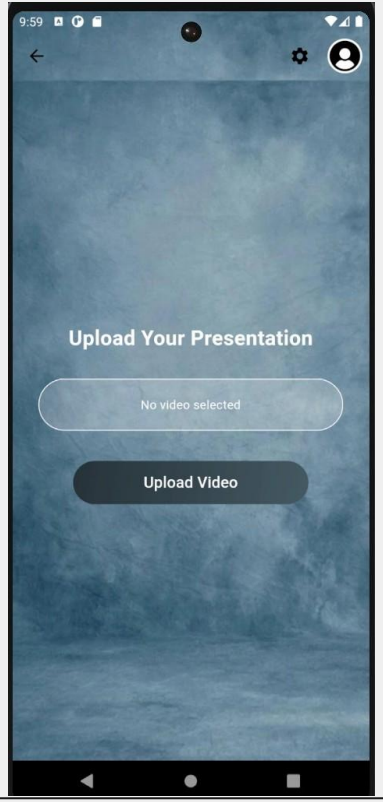


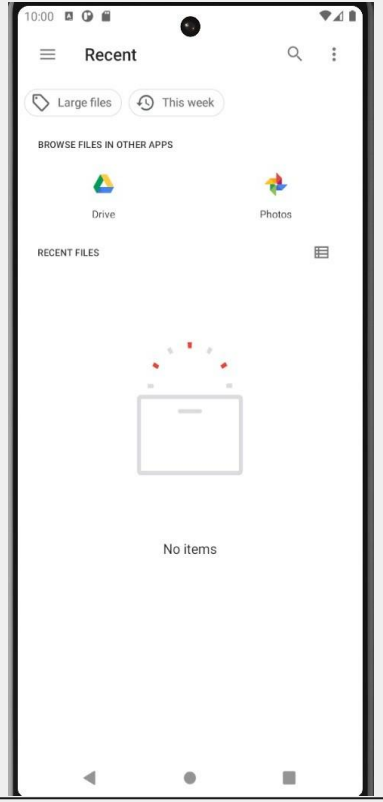


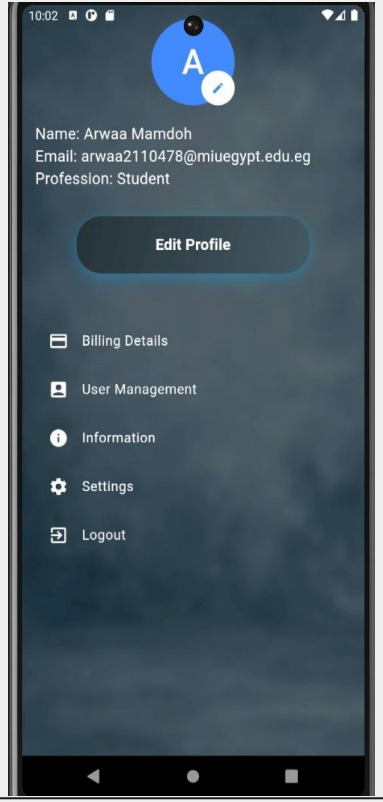


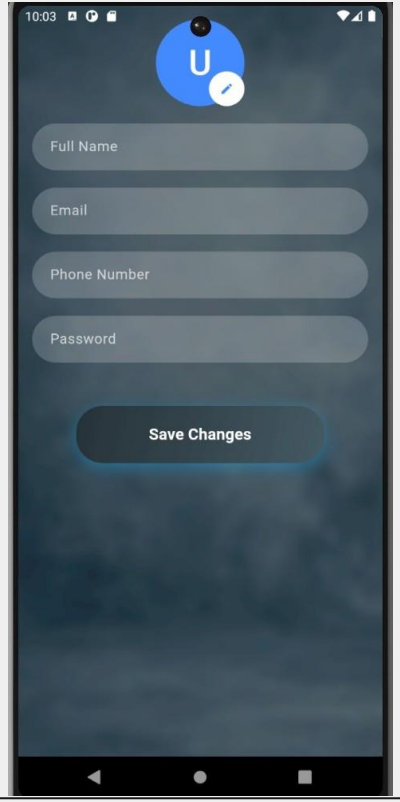


05 USER INTERFACE

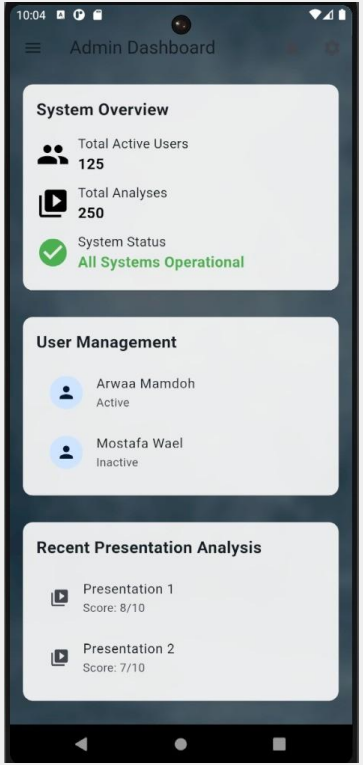


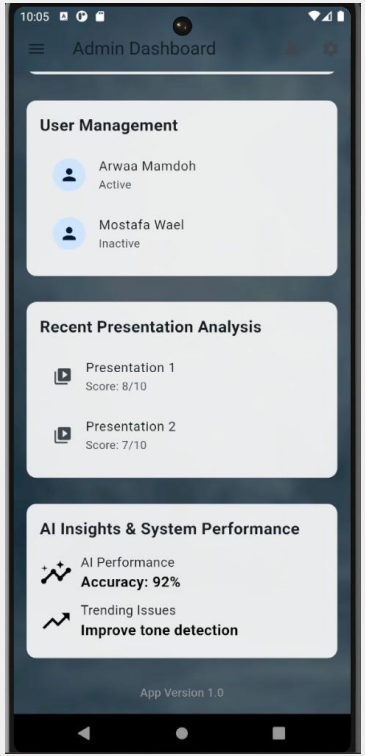


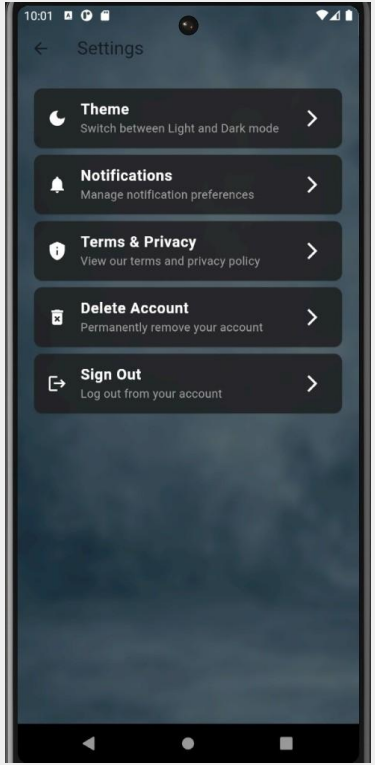




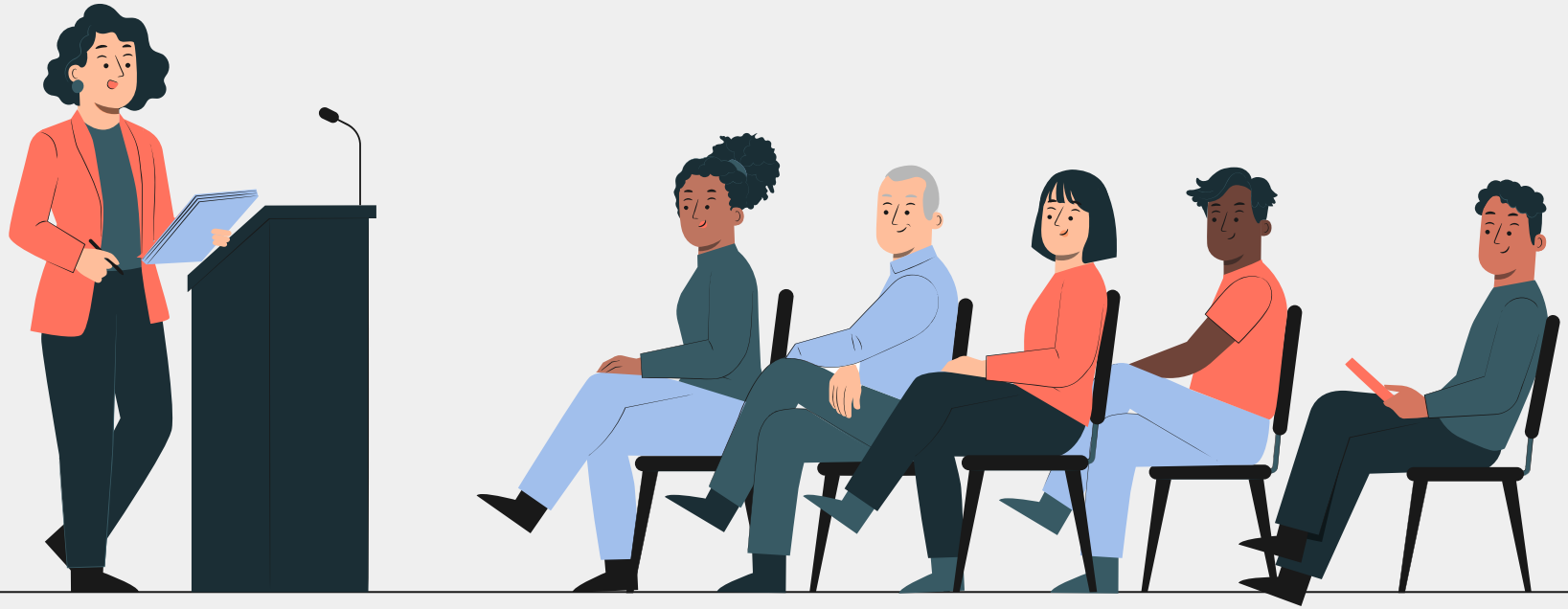
05 USER INTERFACE







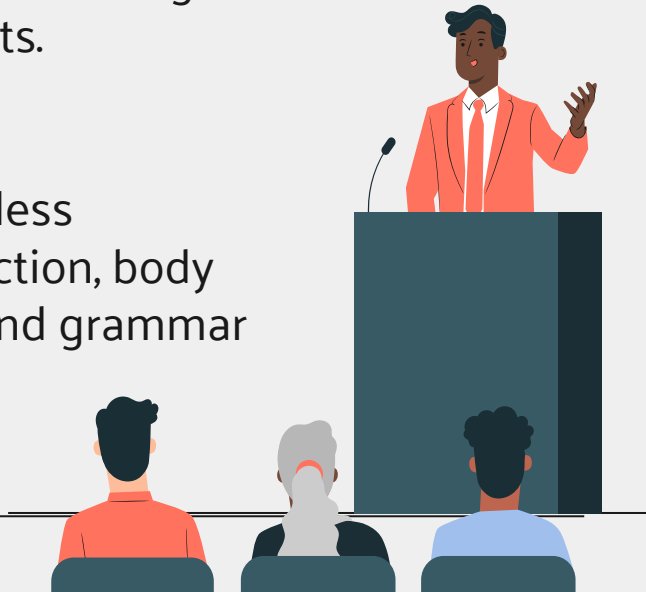
06 WHAT'S NEXT



06 WHAT'S NEXT

- **Improving AI Accuracy** : Refining the AI models for better speech analysis, tone detection, and body language evaluation through additional training and fine-tuning on diverse datasets.

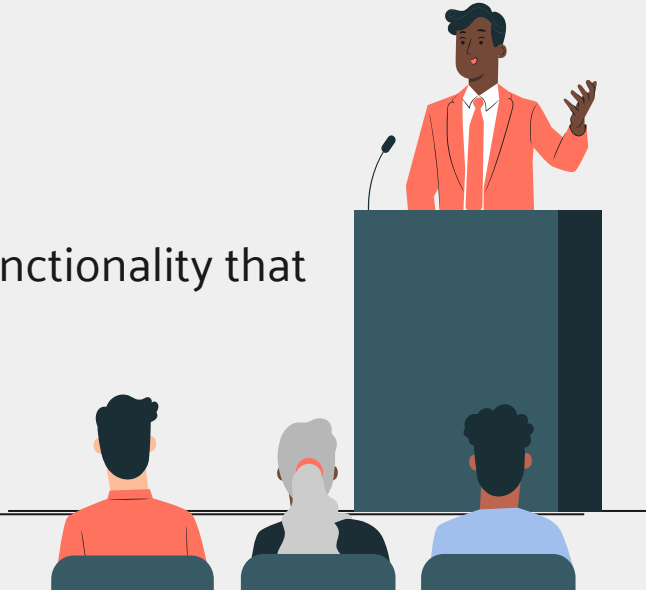
- **Integration of All Models Together:** Ensuring seamless communication between speech analysis, tone detection, body language evaluation, facial expression recognition, and grammar analysis models.



06 WHAT'S NEXT

- **Interaction With Audience :** Upon integrating our models into a unified system, it will have the capability to detect instances of speaker interruption and provide feedback.

- **Application Functionality** : Handling the backend functionality that powers the app.

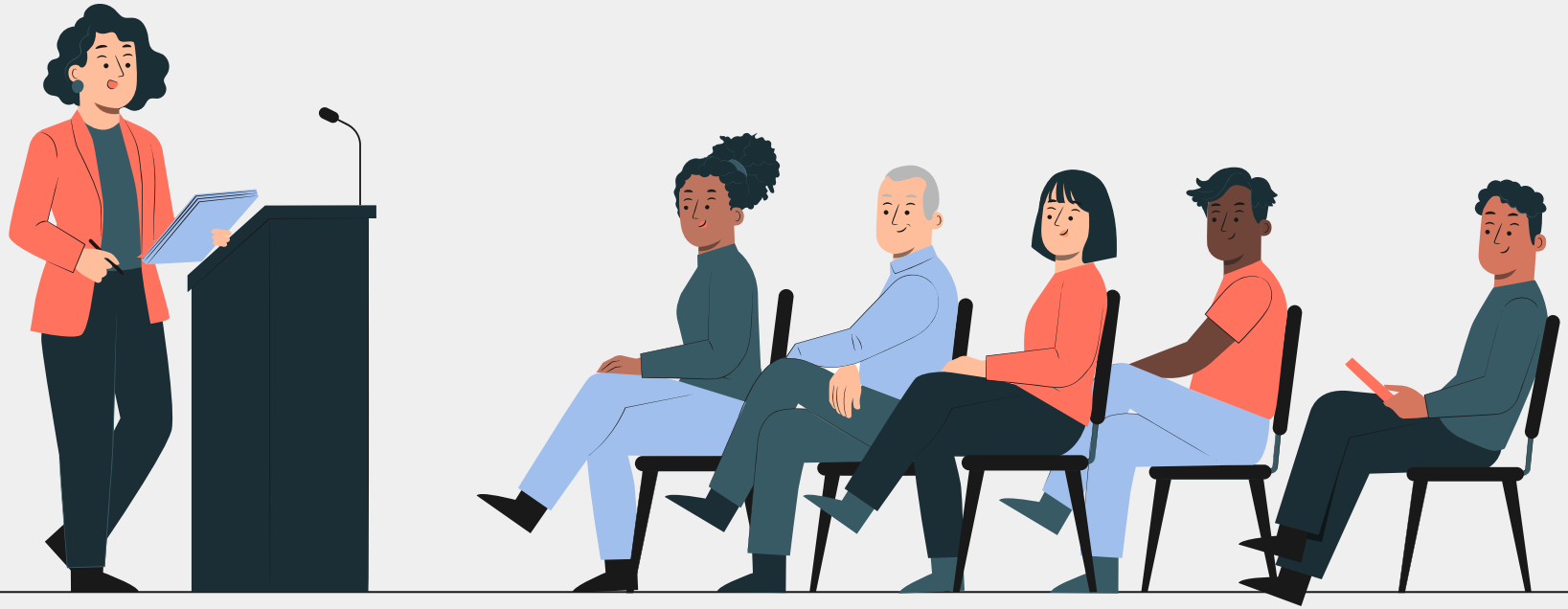


06 WHAT'S NEXT

- **Integration between Ai and App :** integrate the AI models with the app via APIs, enabling seamless communication between the app's frontend and backend . These APIs will handle data processing, analysis, and feedback generation in real-time.

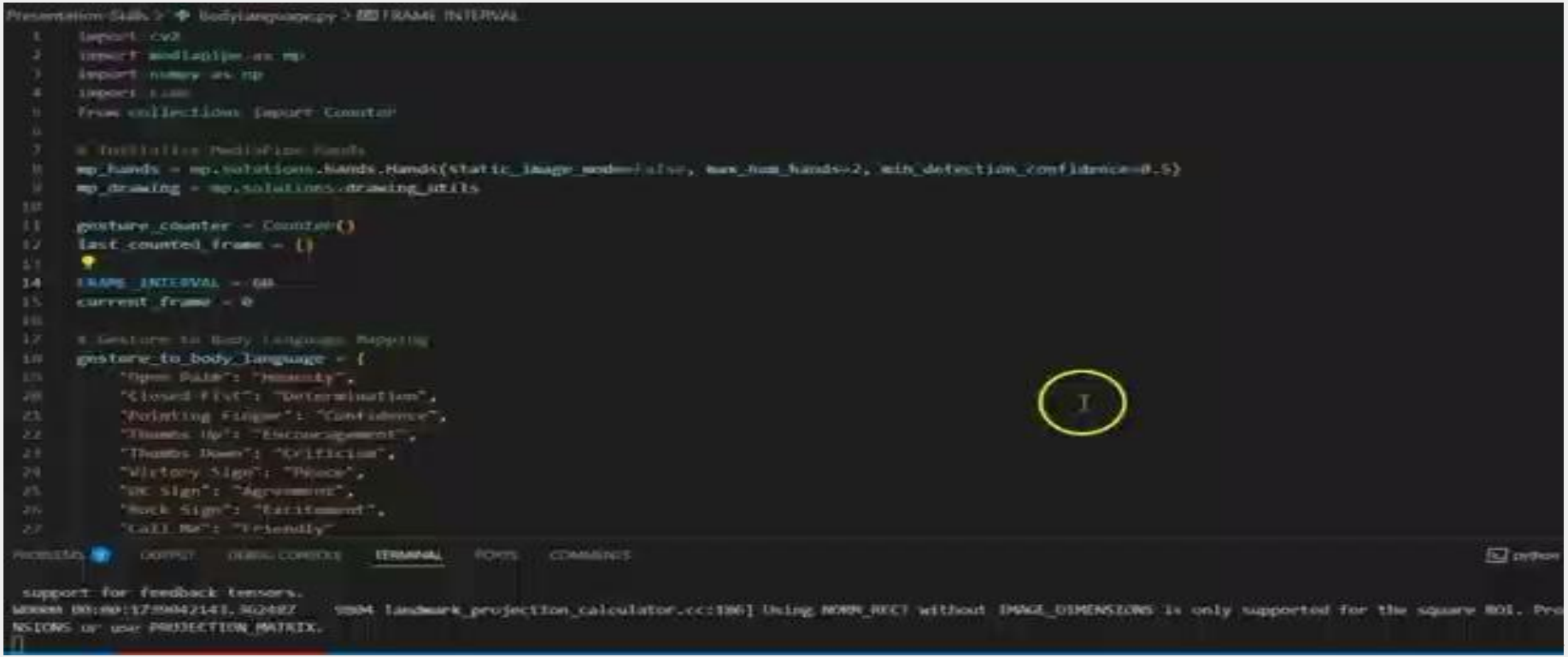
- **Application Deployment :** The app will be deployed on a cloud platform for scalability , It will be available for download on the Play Store and App Store for user accessibility.

07 DEMO



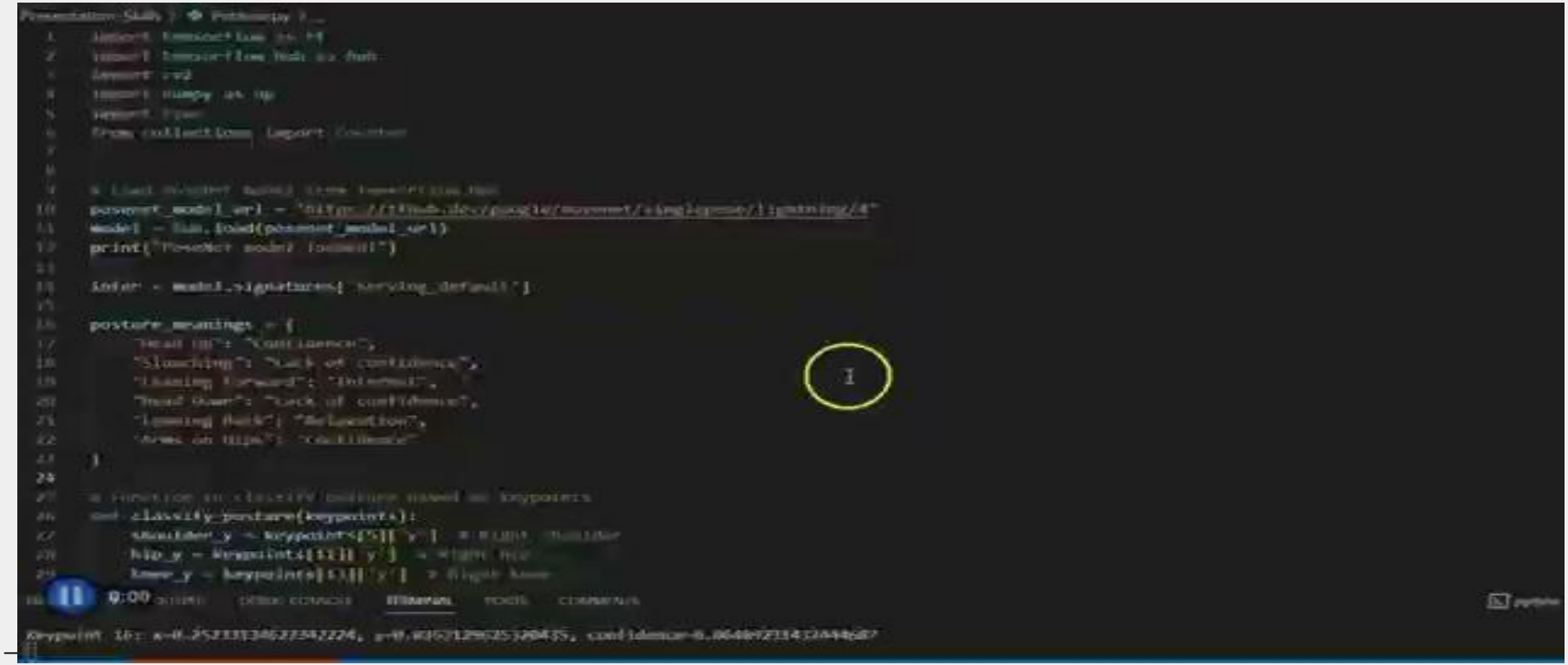
07 DEMO

- **Gesture**



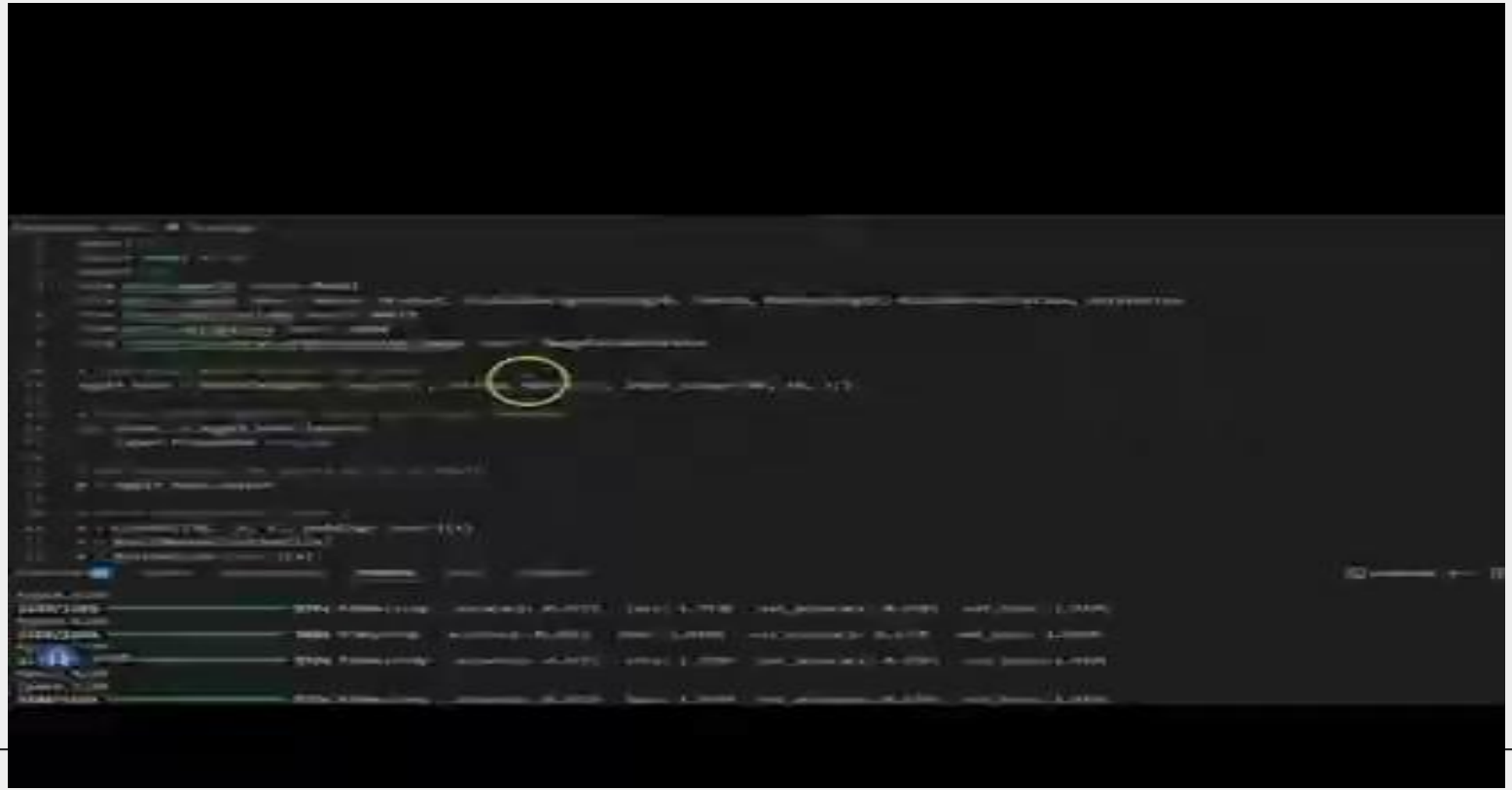
07 DEMO

- **Posture**



07 DEMO

- Facial Emotion



07 DEMO

- **Tone**

File Edit Selection View Go Run Terminal Help Presentation-Skills-1

modified.py 1

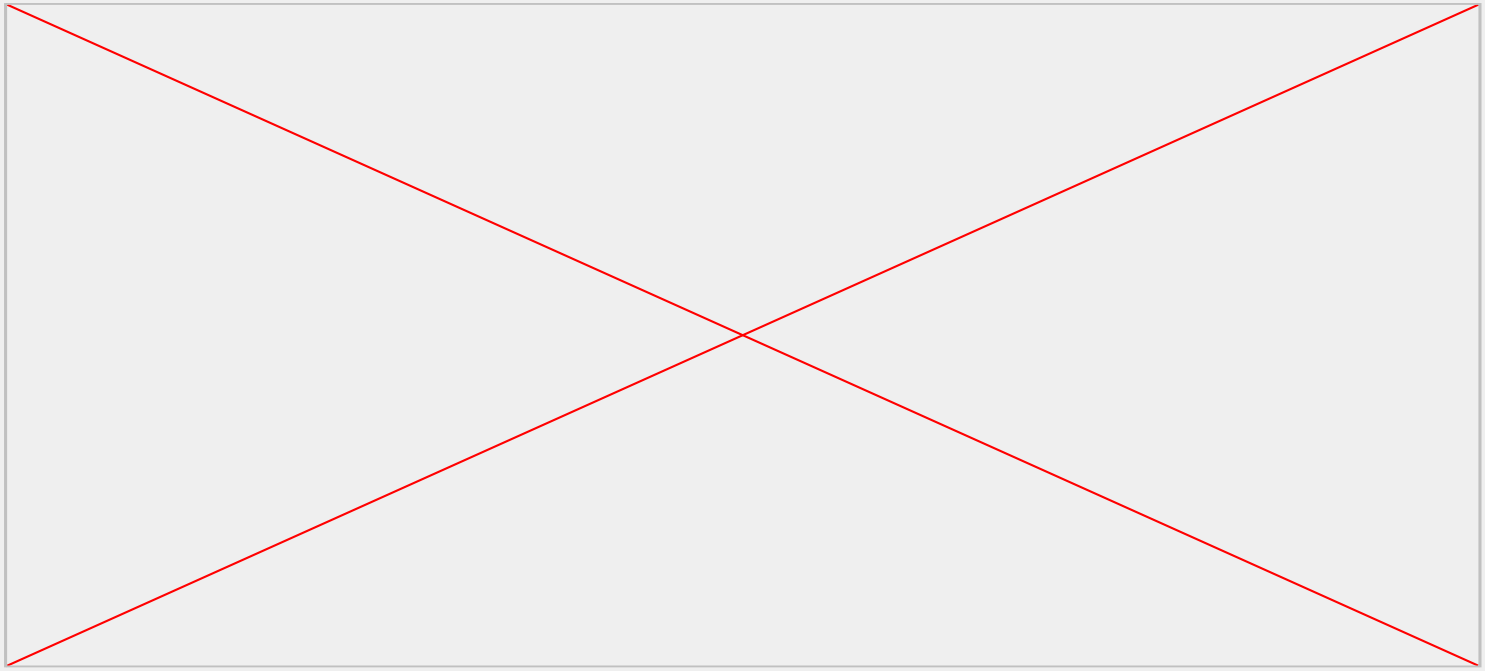
C: > Users > MOUSTAFA > Downloads > modified.py > ...

```python
import numpy as np
import pickle
import librosa
import soundfile as sf
from keras.models import load_model
from sklearn.preprocessing import StandardScaler

#  Load the trained model from .h5 (Corrected path)
model_path = r"C:\Users\MOUSTAFA\Downloads\CNN_Model_Weights (1).h5"
loaded_model = load_model(model_path)

#  Compile the model manually (if needed)
loaded_model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
print(" Model loaded and compiled successfully!")

# Load the scaler (Pickle file)
scaler_path = r"D:\Speech Emotion Recognition-20250207T115414Z-001\D__Speech Emotion Recognition-20250207T115414Z-001_Speech Emotion Recognition_Scaler.pickle"
with open(scaler_path, "rb") as file:
    scaler = pickle.load(file)
print("Scaler loaded successfully!")

#  Load the encoder (Pickle file)
encoder_path = r"D:\Speech Emotion Recognition-20250207T115414Z-001\Speech Emotion Recognition\Encoder.pickle"
with open(encoder_path, "rb") as file:
    encoder = pickle.load(file)
print(" Encoder loaded successfully!")

#  Define Model Expected Input Shape
EXPECTED_FEATURES = 2376  # Change this to match your model's expected input size

#  Function to extract features
def compute_zcr(audio_data, frame_length=2048, hop_length=512):
    return np.squeeze(librosa.feature.zero_crossing_rate(audio_data, frame_length=frame_length, hop_length=hop_length))

def compute_rmse(audio_data, frame_length=2048, hop_length=512):
    return np.squeeze(librosa.feature.rms(y=audio_data, frame_length=frame_length, hop_length=hop_length))
```

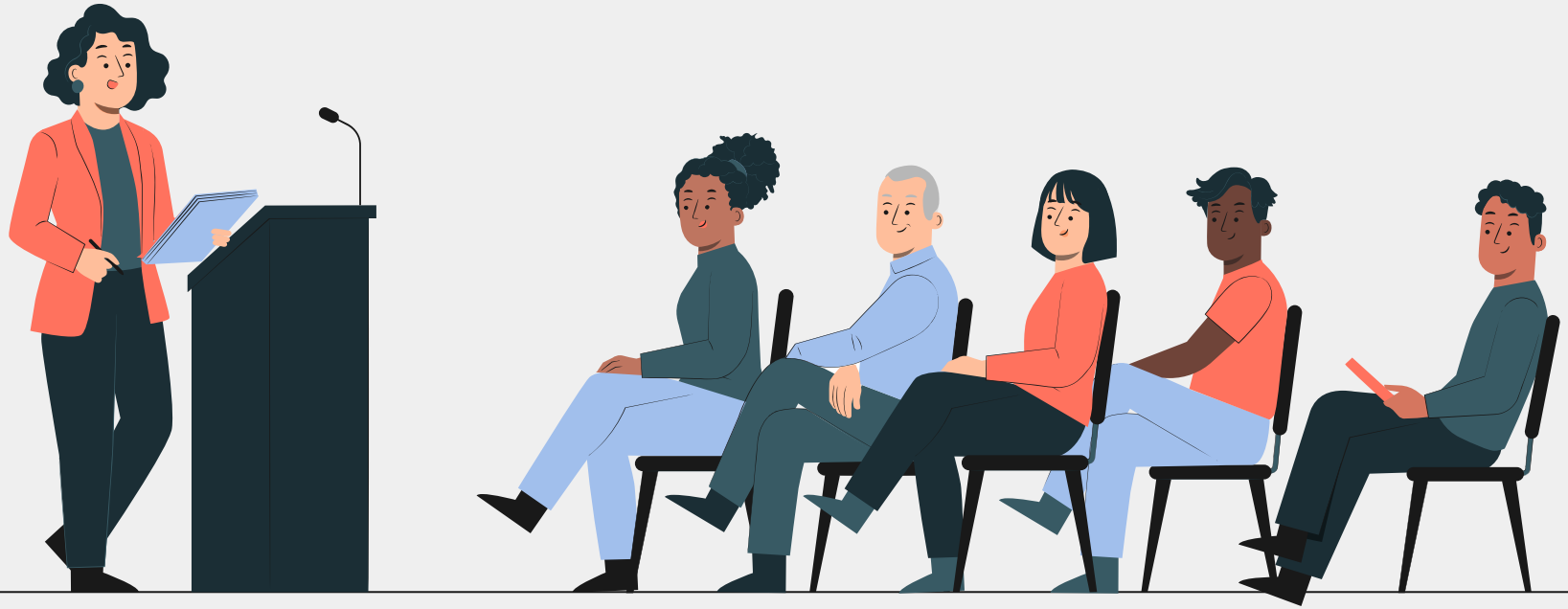
main* 4↓ 0↑ ⊗ 0 ⚠ 24 ⓘ 116 Ln 96, Col 63 Spaces: 4 UTF-8 CRLF {} Python 3.11.0 Windows (windows-x64)

07 DEMO

- **Filler words & Pronunciation**



08 REFERENCES



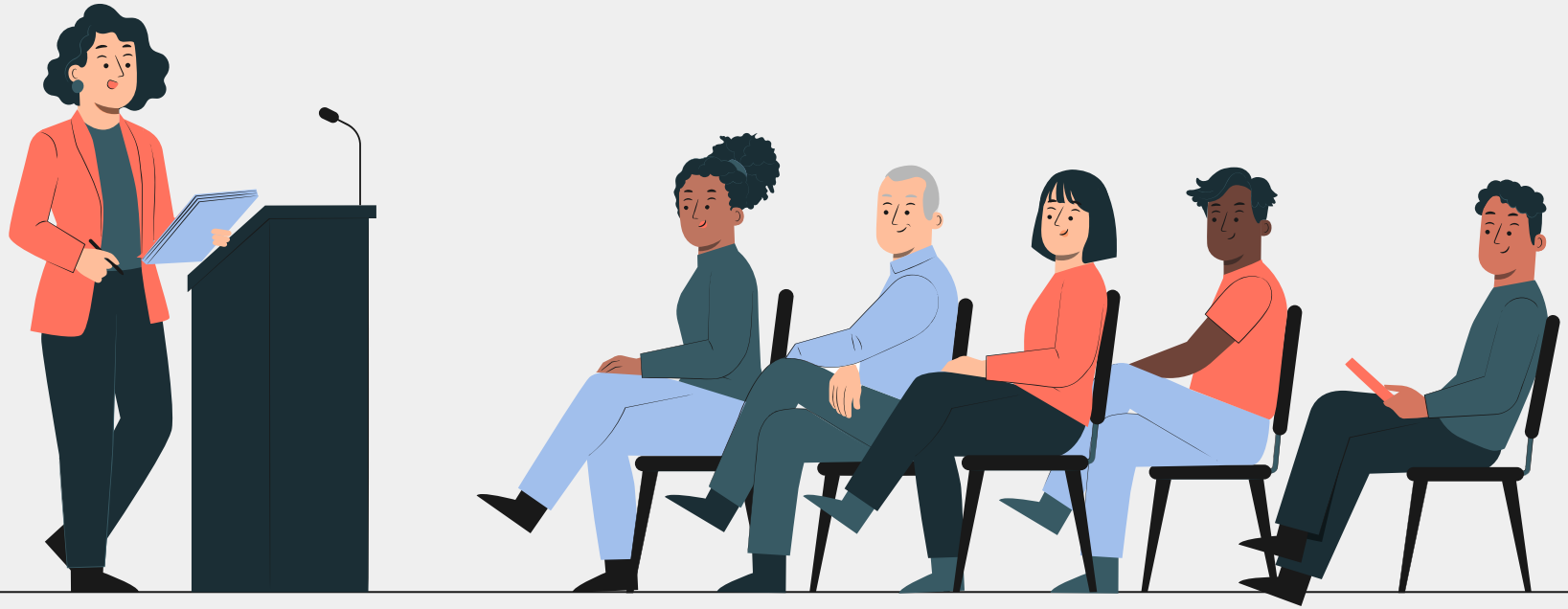
08 REFERENCES

https://link.springer.com/article/10.1007/s42452-024-06117-4

https://ouci.dntb.gov.ua/en/works/422gbgX4/

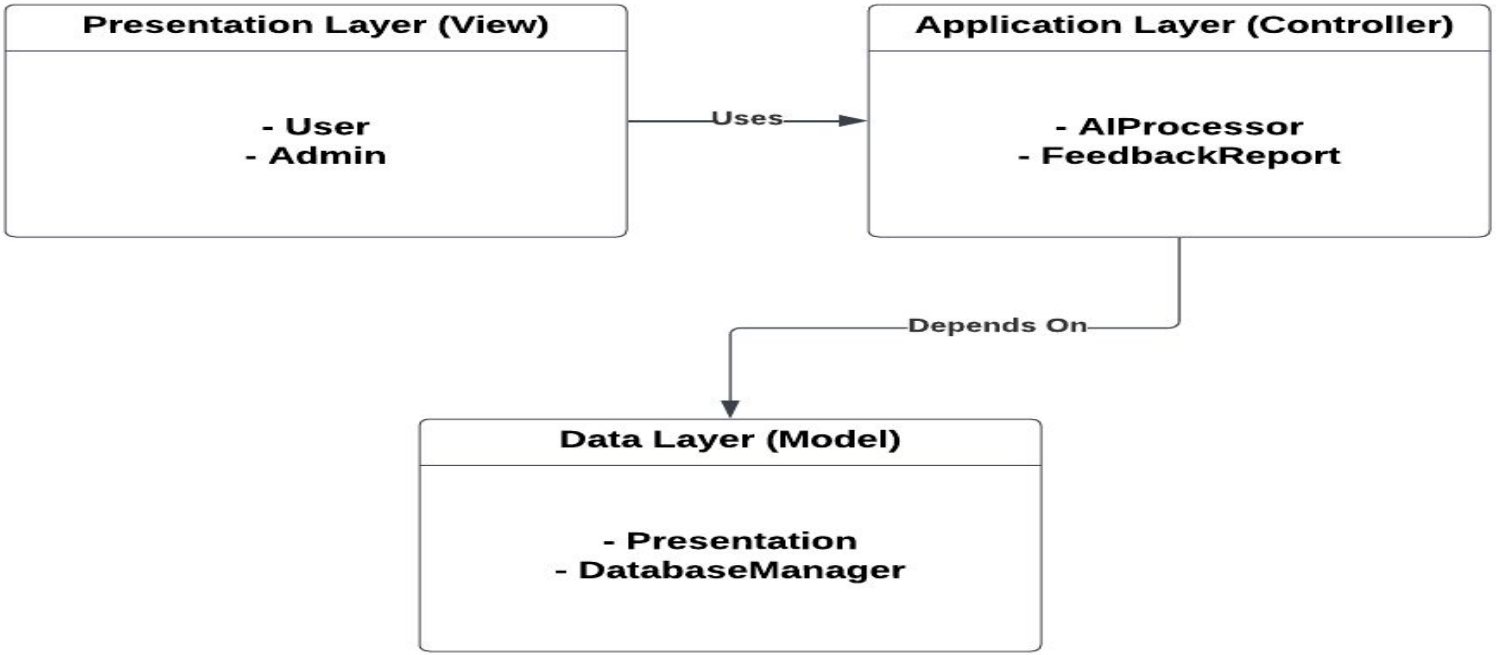
https://journals.sagepub.com/doi/full/10.1177/21582440231193818

09 APPENDIX



09 APPENDIX

UML Package Diagram



09 APPENDIX

Dataset Description table:

Dataset Name	Number of Samples	Number of Classes	Format
SAVEE	480	7	.wav
TESS	2800	7	.wav
RAVDESS	1440	7	.wav
CREMA-D	7442	6	.wav
FER2013	35,887	7	.png
AffectNet	1M+	8	.jpg

Thanks!

Do you have any questions?